Anatomy of the Loadable Kernel Module (LKM)

Adrian Huang

Agenda

- From ‘insmod’ command
- Call path for LKM’s init function
- ‘.gnu.linkonce.this_module’ section
- Deep Dive into call path
- modinfo

Note

- Kernel source: 5.10
- Mainly focus on the ‘init’ function call path

From `insmod` command

Hello World Kernel Module

```c
static int __init hello_init(void)
{
    pr_info("Hello, world\n");
    pr_info("The process is \"%s\" (pid %i)\n",
                current->comm, current->pid);
    return 0;
}

static void __exit hello_exit(void)
{
    printk(KERN_INFO "Goodbye\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

strace

```
# strace insmod hello.ko
execve("/usr/sbin/insmod", ["insmod", "hello.ko"], 0x7ffcfaf01e88 /* 31 vars */)
 = 0
...
openat(AT_FDCWD, "/root/adrian/hello_world/hello.ko", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1", 6)                       = 6
lseek(3, 0, SEEK_SET)                           = 0
fstat(3, {st_mode=S_IFREG|0644, st_size=230744, ...}) = 0
mmap(NULL, 230744, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7faecc373000
finit_module(3, "", 0)                          = 0
munmap(0x7faecc373000, 230744)                  = 0
close(3)                                        = 0
exit_group(0)                                   = ?
+++ exited with 0 +++
```

finit_module() system call loads an ELF image into kernel space

From `insmod` command

Hello World Kernel Module

```c
static int __init hello_init(void)
{
    pr_info("Hello, world\n");
    pr_info("The process is \"%s\" (pid %i)\n",
                current->comm, current->pid);
    return 0;
}

static void __exit hello_exit(void)
{
    printk(KERN_INFO "Goodbye\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

strace

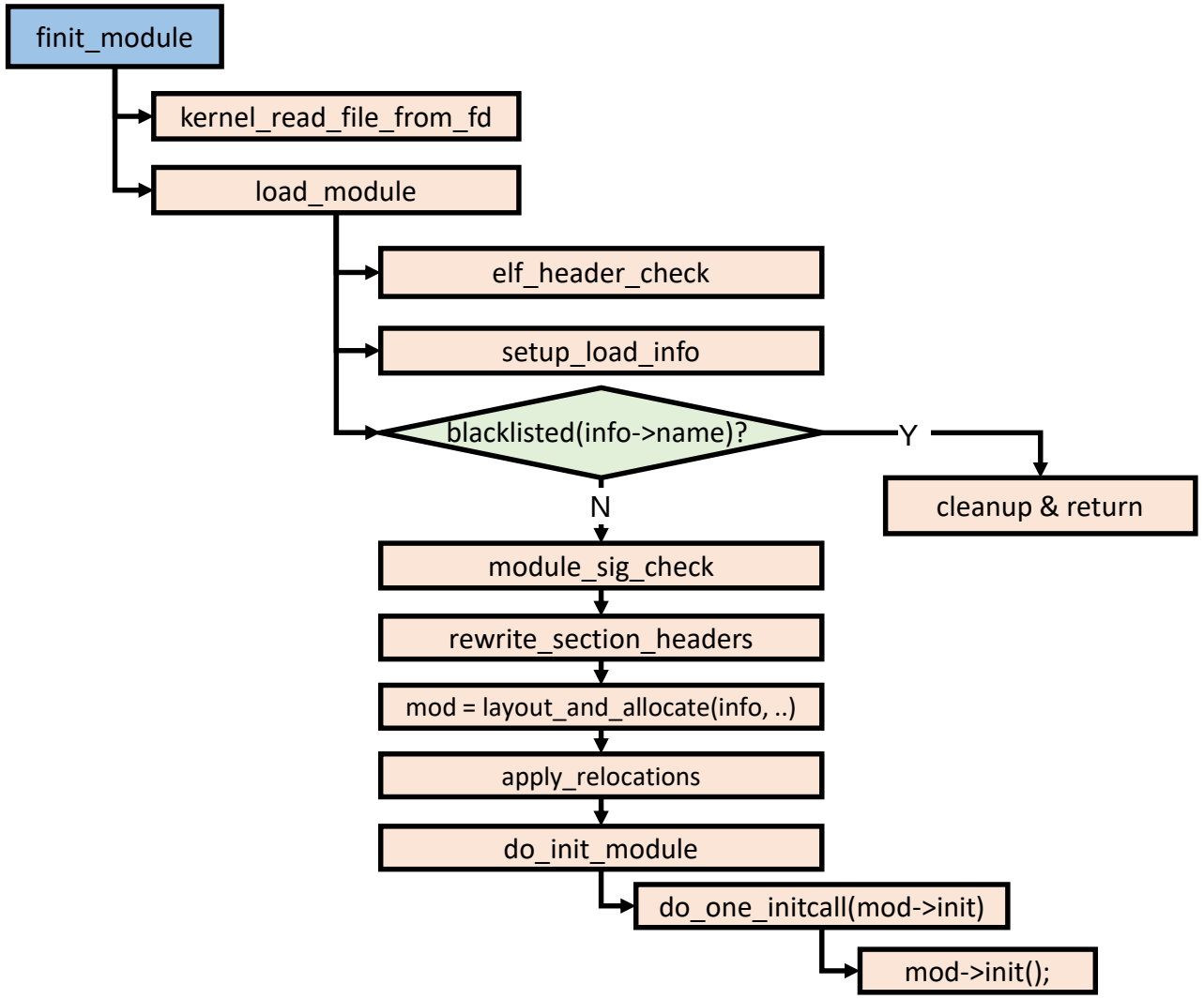
```
# strace insmod hello.ko
execve("/usr/sbin/insmod", ["insmod", "hello.ko"], 0x7ffcfaf01e88 /* 31 vars */)
 = 0
...
openat(AT_FDCWD, "/root/adrian/hello_world/hello.ko", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1", 6)                       = 6
lseek(3, 0, SEEK_SET)                           = 0
fstat(3, {st_mode=S_IFREG|0644, st_size=230744, ...}) = 0
mmap(NULL, 230744, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7faecc373000
finit_module(3, "", 0)                          = 0
munmap(0x7faecc373000, 230744)                  = 0
close(3)                                        = 0
exit_group(0)                                   = ?
+++ exited with 0 +++
```

finit_module()

- Load an ELF image into kernel space
- Perform symbol relocations
- Initialize module parameters to values
- Run the module's init function

finit_module() system call loads an ELF image into kernel space

Call path for LKM's init function



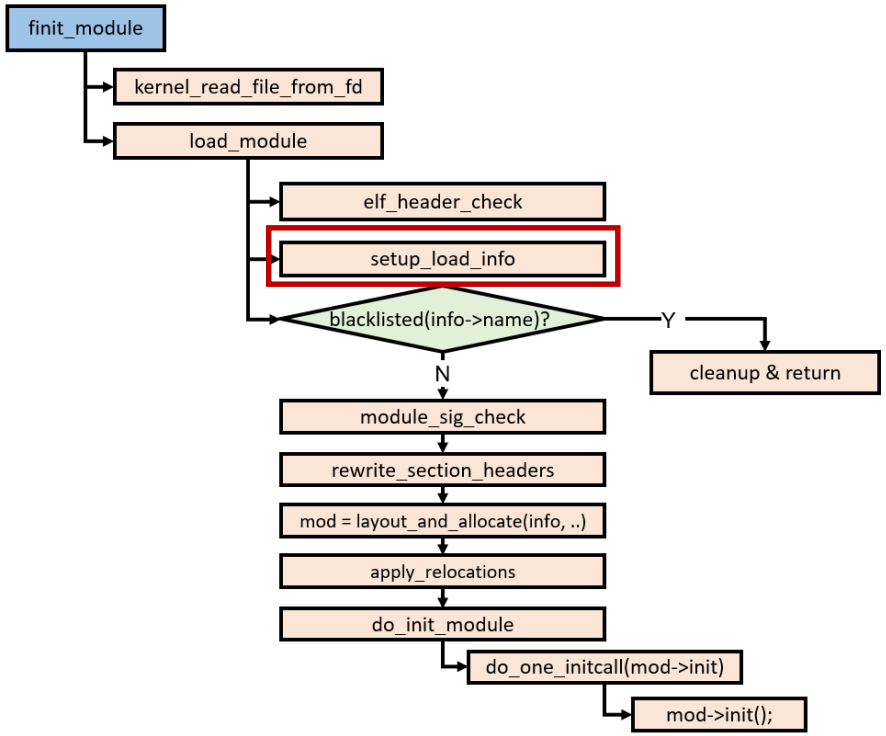

```
struct load_info {
        const char *name;
        /* pointer to module in temporary copy, freed at end of load_module()
*/
        struct module *mod;
        Elf_Ehdr *hdr;
        unsigned long len;
        Elf_Shdr *sechdrs;
        char *secstrings, *strtab;
        unsigned long symoffs, stroffs, init_typeoffs, core_typeoffs;
        struct _ddebug *debug;
        unsigned int num_debug;
        bool sig_ok;
#ifdef CONFIG_KALLSYMS
        unsigned long mod_kallsyms_init_off;
#endif
        struct {
                unsigned int sym, str, mod, vers, info, pcpu;
        } index;
};
```

```
struct module {
        ...

        /* Startup function. */
        int (*init)(void);

        ...

        /* Destruction function. */
        void (*exit)(void);
} ____cacheline_aligned __randomize_layout;
```

Analysis

- Key: mod->init()
- How to assign the address of mod->init()?

‘.gnu.linkonce.this_module’ section (1/6)




```
static int setup_load_info(struct load_info *info, int flags)
{
        ...
        info->index.mod = find_sec(info, ".gnu.linkonce.this_module");
        if (!info->index.mod) {
                pr_warn("%s: No module found in object\n",
                        info->name ?: "(missing .modinfo section or name field
)");
                return -ENOEXEC;
        }
        /* This is temporary: point mod into copy of data. */
        info->mod = (void *)info->hdr + info->sechdrs[info->index.mod].sh_offs
et;
        ...

        return 0;
}
```

```
# readelf -a hello.ko

Section Headers:
  [Nr] Name              Type             Address           Offset
       Size              EntSize          Flags  Link  Info  Align
  ...
  [18] .gnu.linkonce.thi PROGBITS         0000000000000000  00000300
       00000000000003c0  0000000000000000  WA       0     0     64
  [19] .rela.gnu.linkonc RELA             0000000000000000  0001f178
       0000000000000030  0000000000000018   I      37    18     8
...
Relocation section '.rela.gnu.linkonce.this_module' at offset 0x1f178 contains 2
 entries:
  Offset          Info           Type           Sym. Value    Sym. Name + Addend
000000000150  002b00000001 R_X86_64_64       0000000000000000 init_module + 0
000000000380  002900000001 R_X86_64_64       0000000000000000 cleanup_module + 0
```

'.gnu.linkonce.this_module' section (2/6)

```
# readelf -a hello.ko

Section Headers:
  [Nr] Name              Type             Address           Offset
       Size              EntSize          Flags  Link  Info  Align
  ...
  [18] .gnu.linkonce.thi PROGBITS         0000000000000000  00000300
       00000000000003c0  0000000000000000  WA       0     0     64
  [19] .rela.gnu.linkonc RELA             0000000000000000  0001f178
       0000000000000030  0000000000000018   I      37    18     8
...
Relocation section '.rela.gnu.linkonce.this_module' at offset 0x1f178 contains 2
 entries:
  Offset          Info           Type           Sym. Value    Sym. Name + Addend
000000000150  002b00000001 R_X86_64_64       0000000000000000 init_module + 0
000000000380  002900000001 R_X86_64_64       0000000000000000 cleanup_module + 0
```

```
crash> struct module -o -x
struct module {
    [0x0] enum module_state state;
    [0x8] struct list_head list;
   [0x18] char name[56];
   [0x50] struct module_kobject mkobj;
   ...
  [0x150] int (*init)(void);
  ...
  [0x380] void (*exit)(void);
  [0x388] atomic_t refcnt;
  [0x390] struct error_injection_entry *ei_funcs;
  [0x398] unsigned int num_ei_funcs;
}
SIZE: 0x3c0
```

```
# xxd hello.ko
00000300: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000310: 0000 0000 0000 0000 6865 6c6c 6f00 0000  ........hello...
00000320: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000330: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000340: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000350: 0000 0000 0000 0000 0000 0000 0000 0000  ................
...
00000450: 0000 0000 0000 0000 0000 0000 0000 0000  ................
...
00000680: 0000 0000 0000 0000 0000 0000 0000 0000  ................
...
```

‘.gnu.linkonce.this_module’ section (3/6)

```
# readelf -a hello.ko

Section Headers:
  [Nr] Name              Type             Address           Offset
       Size              EntSize          Flags  Link  Info  Align
  ...
  [18] .gnu.linkonce.thi PROGBITS         0000000000000000  00000300
       00000000000003c0  0000000000000000  WA       0     0     64
  [19] .rela.gnu.linkonc RELA             0000000000000000  0001f178
       0000000000000030  0000000000000018   I      37    18     8
...
Relocation section '.rela.gnu.linkonce.this_module' at offset 0x1f178 contains 2
 entries:
  Offset          Info           Type           Sym. Value    Sym. Name + Addend
000000000150  002b00000001 R_X86_64_64       0000000000000000 init_module + 0
000000000380  002900000001 R_X86_64_64       0000000000000000 cleanup_module + 0
```

```
crash> struct module -o -x
struct module {
    [0x0] enum module_state state;
    [0x8] struct list_head list;
   [0x18] char name[56];
   [0x50] struct module_kobject mkobj;
   ...
  [0x150] int (*init)(void);
  ...
  [0x380] void (*exit)(void);
  [0x388] atomic_t refcnt;
  [0x390] struct error_injection_entry *ei_funcs;
  [0x398] unsigned int num_ei_funcs;
}
SIZE: 0x3c0
```

```
# xxd hello.ko
00000300: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000310: 0000 0000 0000 0000 6865 6c6c 6f00 0000  ........hello...
00000320: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000330: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000340: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000350: 0000 0000 0000 0000 0000 0000 0000 0000  ................
...
00000450: 0000 0000 0000 0000 0000 0000 0000 0000  ................
...
00000680: 0000 0000 0000 0000 0000 0000 0000 0000  ................
...
```

'.gnu.linkonce.this_module' section (4/6)

```
static void add_header(struct buffer *b, struct module *mod)
{
        buf_printf(b, "#include <linux/module.h>\n");
        /*
         * Include build-salt.h after module.h in order to
         * inherit the definitions.
         */
        buf_printf(b, "#define INCLUDE_VERMAGIC\n");
        buf_printf(b, "#include <linux/build-salt.h>\n");
        buf_printf(b, "#include <linux/vermagic.h>\n");
        buf_printf(b, "#include <linux/compiler.h>\n");
        buf_printf(b, "\n");
        buf_printf(b, "BUILD_SALT;\n");
        buf_printf(b, "\n");
        buf_printf(b, "MODULE_INFO(vermagic, VERMAGIC_STRING);\n");
        buf_printf(b, "MODULE_INFO(name, KBUILD_MODNAME);\n");
        buf_printf(b, "\n");
        buf_printf(b, "__visible struct module __this_module\n");
        buf_printf(b, "__section(\".gnu.linkonce.this_module\") = {\n");
        buf_printf(b, "\t.name = KBUILD_MODNAME,\n");
        if (mod->has_init)
                buf_printf(b, "\t.init = init_module,\n");
        if (mod->has_cleanup)
                buf_printf(b, "#ifdef CONFIG_MODULE_UNLOAD\n"
                              "\t.exit = cleanup_module,\n"
                              "#endif\n");
```

scripts/mod/modpost.c 2257,29-36

```
#include <linux/module.h>
#define INCLUDE_VERMAGIC
#include <linux/build-salt.h>
#include <linux/vermagic.h>
#include <linux/compiler.h>

BUILD_SALT;

MODULE_INFO(vermagic, VERMAGIC_STRING);
MODULE_INFO(name, KBUILD_MODNAME);

__visible struct module __this_module
__section(".gnu.linkonce.this_module") = {
        .name = KBUILD_MODNAME,
        .init = init_module,
#ifdef CONFIG_MODULE_UNLOAD
        .exit = cleanup_module,
#endif
        .arch = MODULE_ARCH_INIT,
};

#ifdef CONFIG_RETPOLINE
MODULE_INFO(retpoline, "Y");
#endif
```

hello.mod.c

```
extern int init_module(void);
extern void cleanup_module(void);
```

include/linux/module.h

User Space Tool – modpost: Generate a file 'module_name.mod.c' when compiling your kernel module

‘.gnu.linkonce.this_module’ section - Where is ‘init_module()’ definition? (5/6)

Hello World Kernel Module

```
static int __init hello_init(void)
{
    pr_info("Hello, world\n");
    pr_info("The process is \"%s\" (pid %i)\n",
            current->comm, current->pid);
    return 0;
}

static void __exit hello_exit(void)
{
    printk(KERN_INFO "Goodbye\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

module_init() macro

```
#define module_init(initfn) \
        int init_module(void) __attribute__((alias(#initfn)));

module_init(hello_init);

--> after expansion

int init_module(void) __attribute__((alias("hello_init")));
```

__init macro

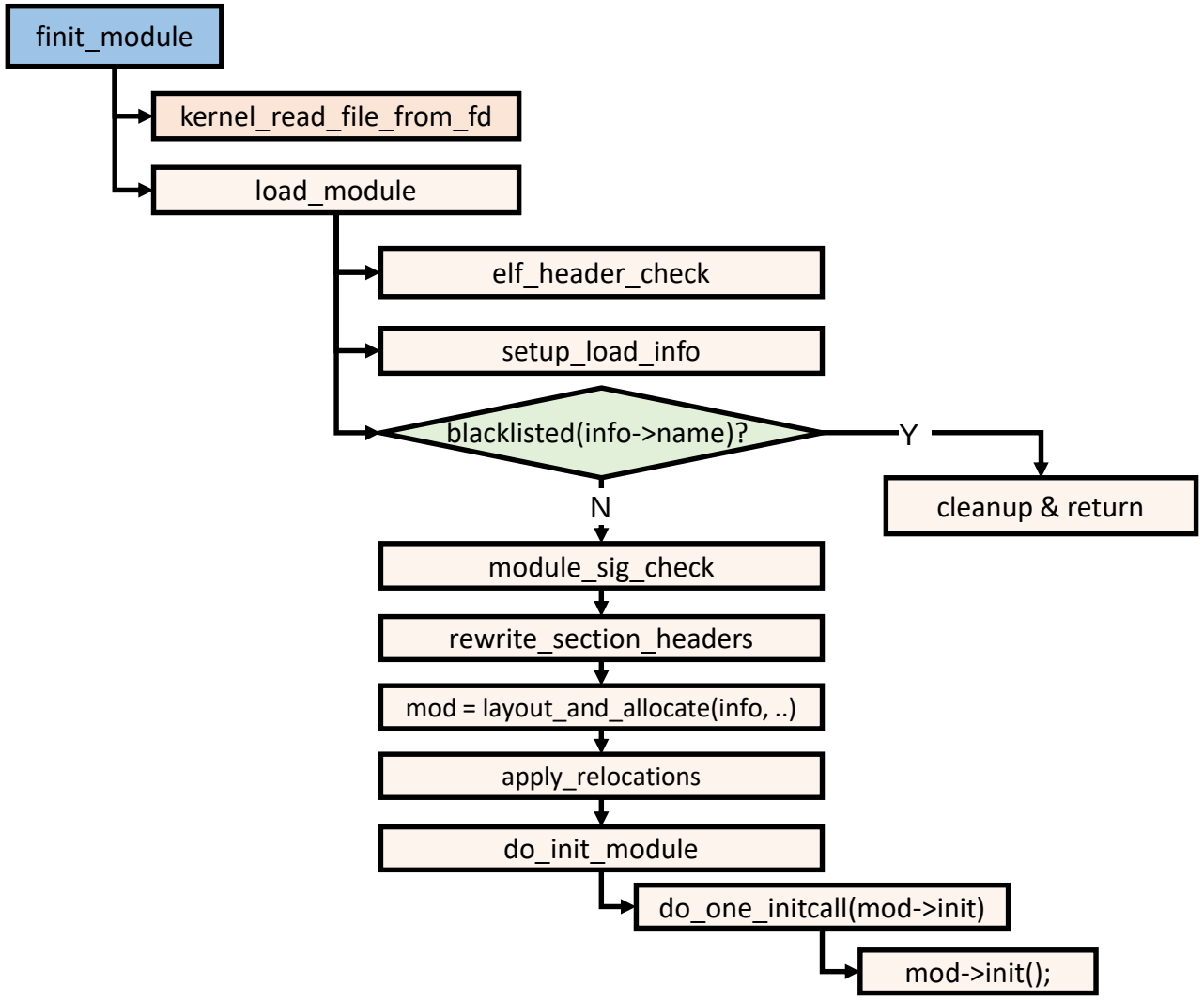
```
#define __init  __section(".init.text")

static int __init hello_init(void)
{
        ...
}

--> after expansion

static int __section(".init.text") hello_init(void)
{
        ...
}
```

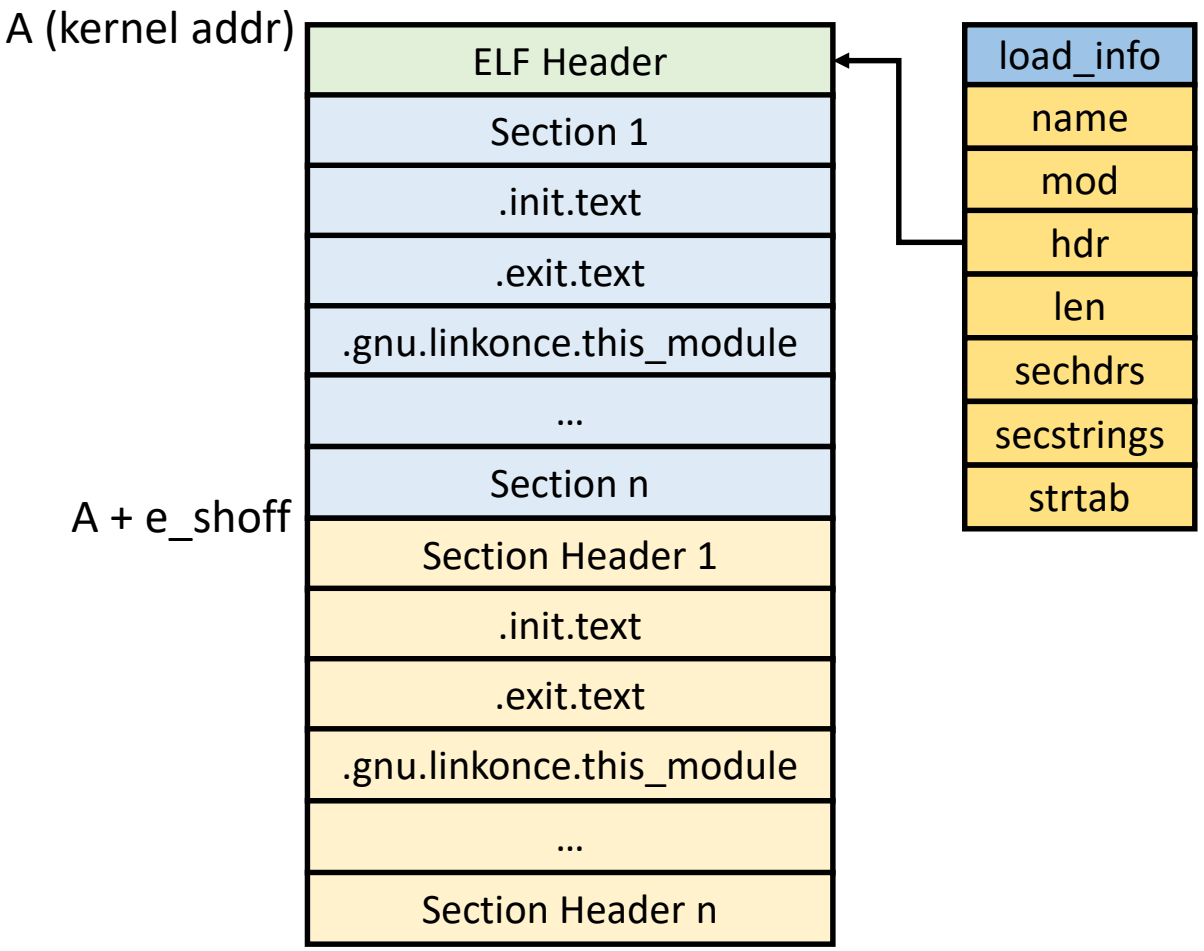
‘.gnu.linkonce.this_module’ section (6/6)

```
# cat /proc/cmdline
BOOT_IMAGE=(hd0,gpt2)/vmlinuz-5.10.0-rc7+ root=/dev/mapper/rh
el-root ro crashkernel=auto resume=/dev/mapper/rhel-swap rd.l
vm.lv=rhel/root rd.lvm.lv=rhel/swap rhgb quiet "dyndbg=file k
ernel/module.c +p"

# insmod hello.ko

# dmesg
[ 3490.698650] finit_module: fd=3, uargs=000000002dbb3adf, fl
ags=0
[ 3490.699981] hello: loading out-of-tree module taints kerne
l.
[ 3490.699987] Core section allocation order:
[ 3490.699991]  .text
[ 3490.699992]  .exit.text
...
[ 3490.700004]  .gnu.linkonce.this_module
[ 3490.700005]  .bss
[ 3490.700007] Init section allocation order:
[ 3490.700008]  .init.text
[ 3490.700011]  .symtab
[ 3490.700013]  .strtab
[ 3490.700039] final section addresses:
...
[ 3490.700044]  0xffffffffc0a00000 .text
[ 3490.700045]  0xffffffffc0915000 .init.text
[ 3490.700047]  0xffffffffc0a00000 .exit.text
...
[ 3490.700057]  0xffffffffc0a02000 .gnu.linkonce.this_module
```

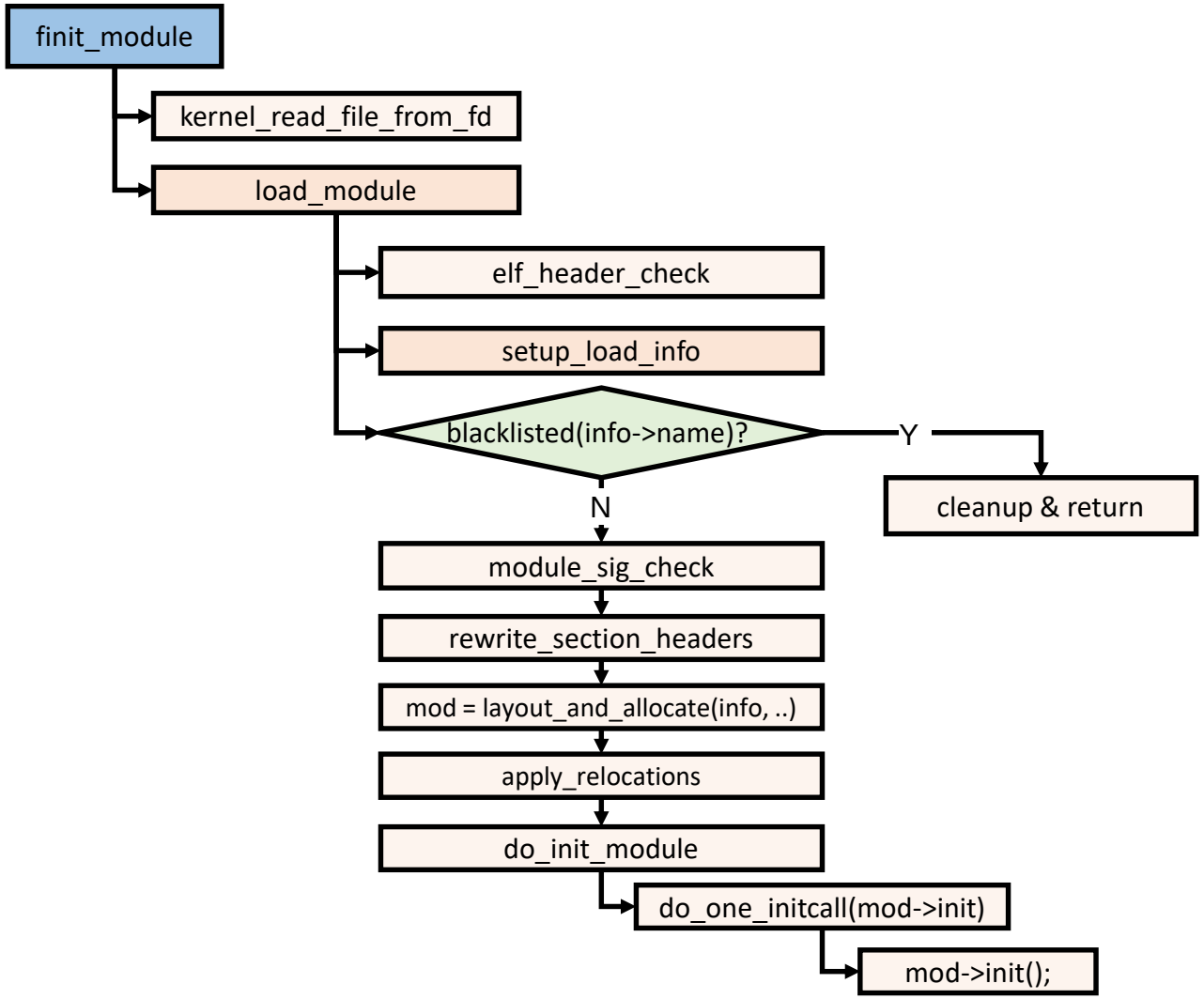
```
crash> struct module 0xffffffffc0a02000
struct module {
  ...
  name = "hello\000\000\000\000\000\000\000\000\000\
000\000\000\000\000\000\000\000\000\000\000\000\000\
000\000\000\000\000\000\000\000\000\000\000\000\000\
000\000\000\000\000\000\000\000\000\000\000\000\000\
000\000",
  mkobj = {
  ...
  init = 0xffffffffc0915000,
  ...
  exit = 0xffffffffc0a00000,
  refcnt = {
    counter = 1
  },
  ei_funcs = 0x0,
  num_ei_funcs = 0
}
```

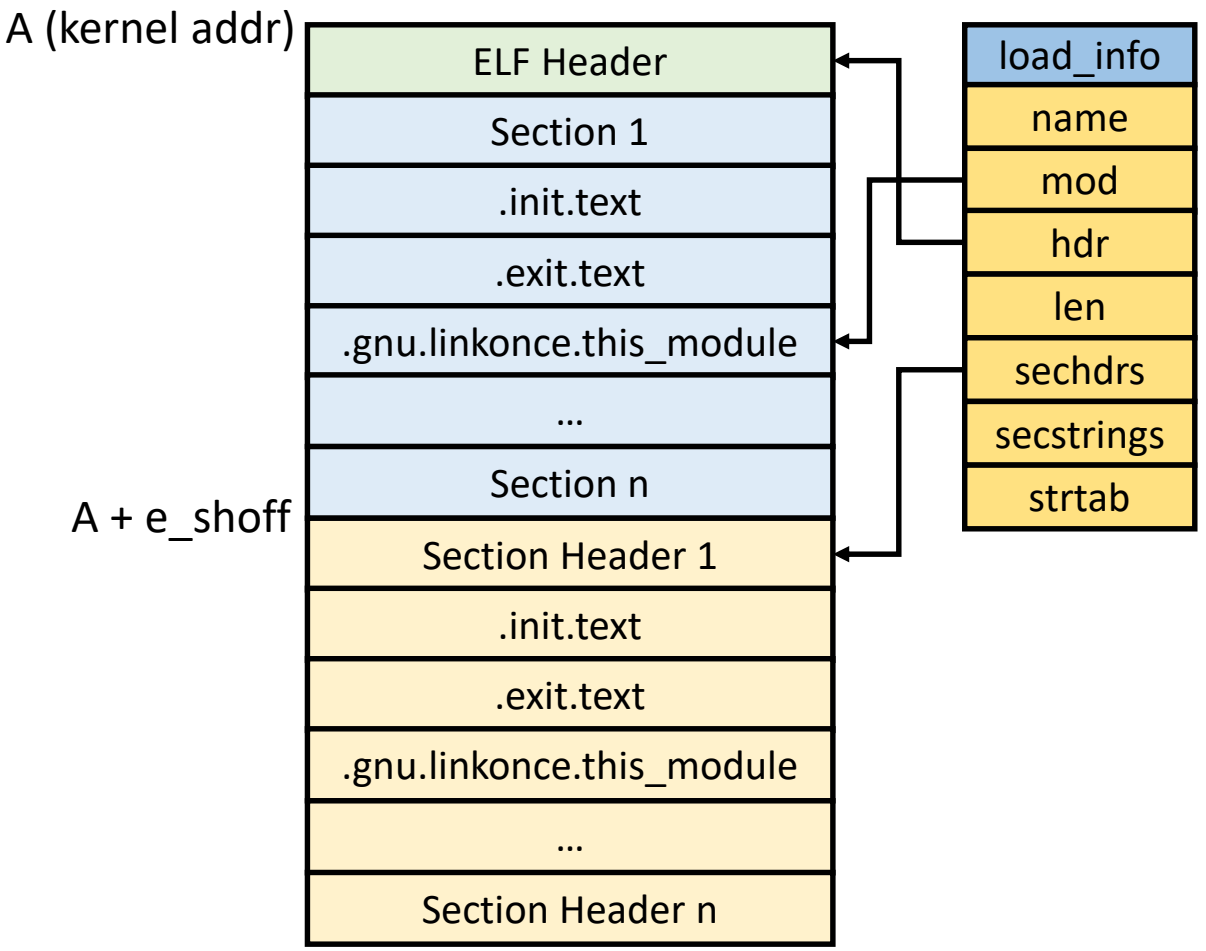
Deep Dive into call path (1/7)



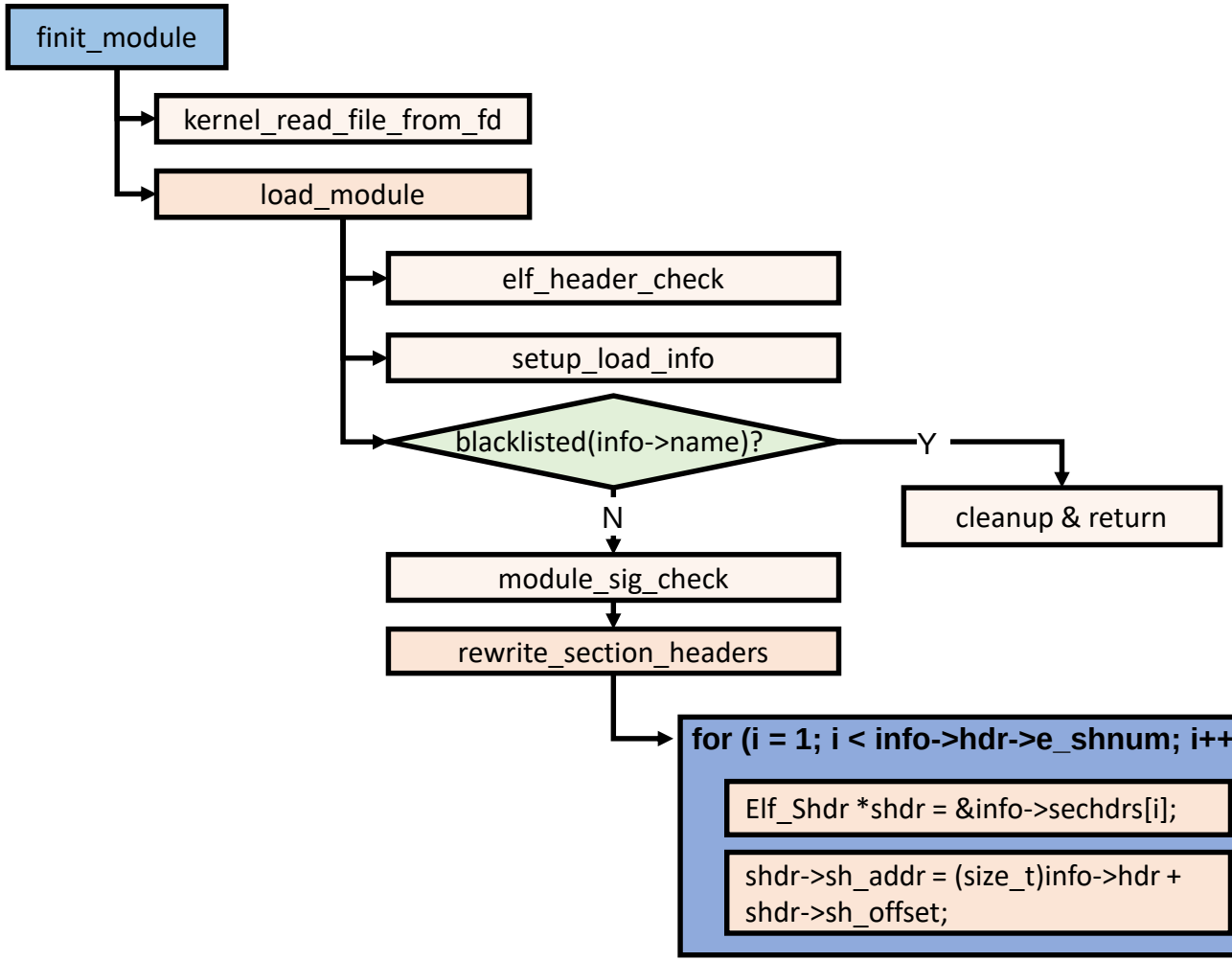


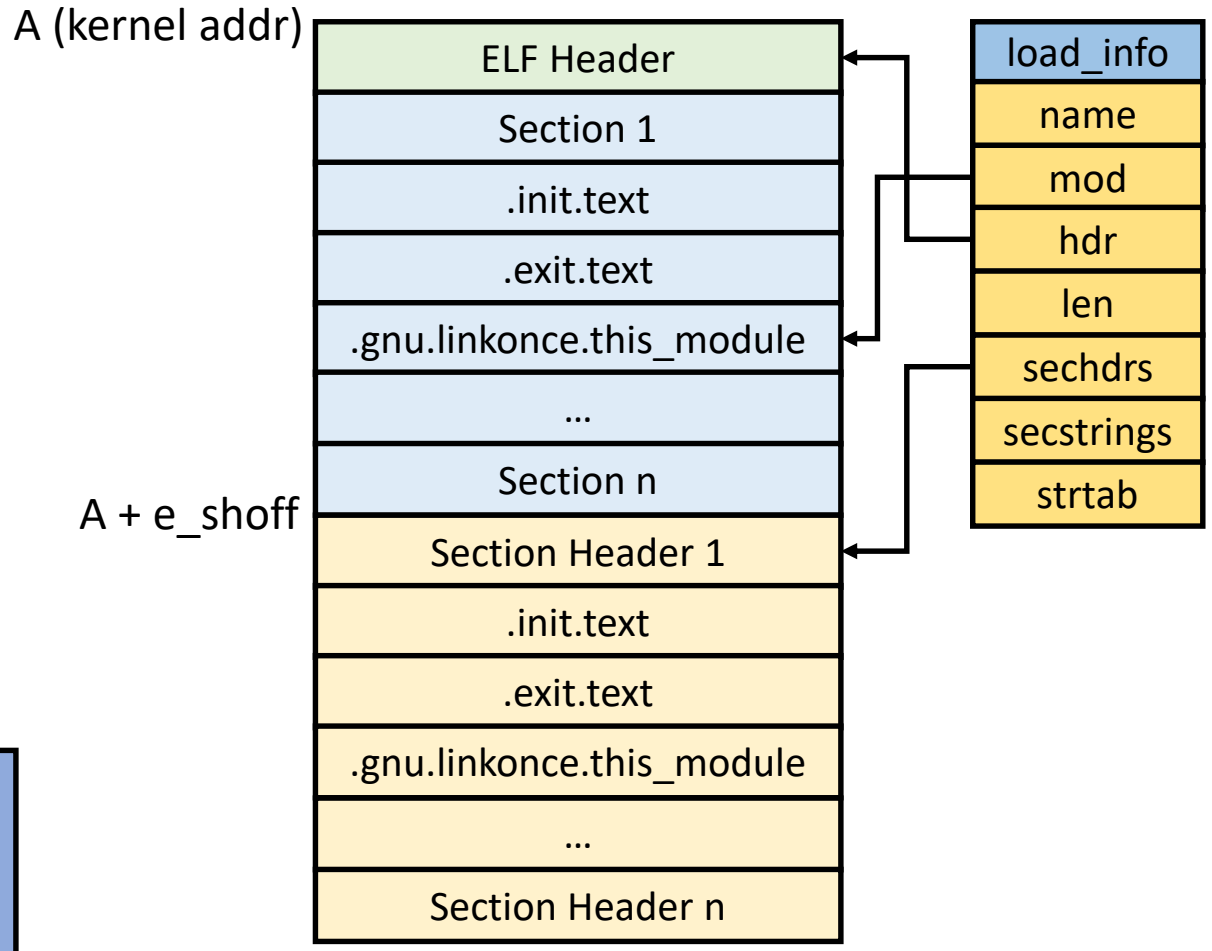
Deep Dive into call path (2/7)





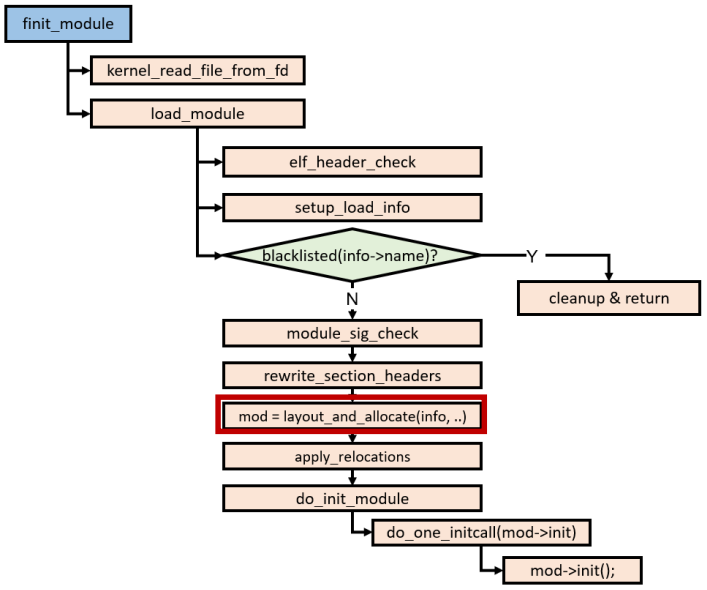
Deep Dive into call path (3/7)

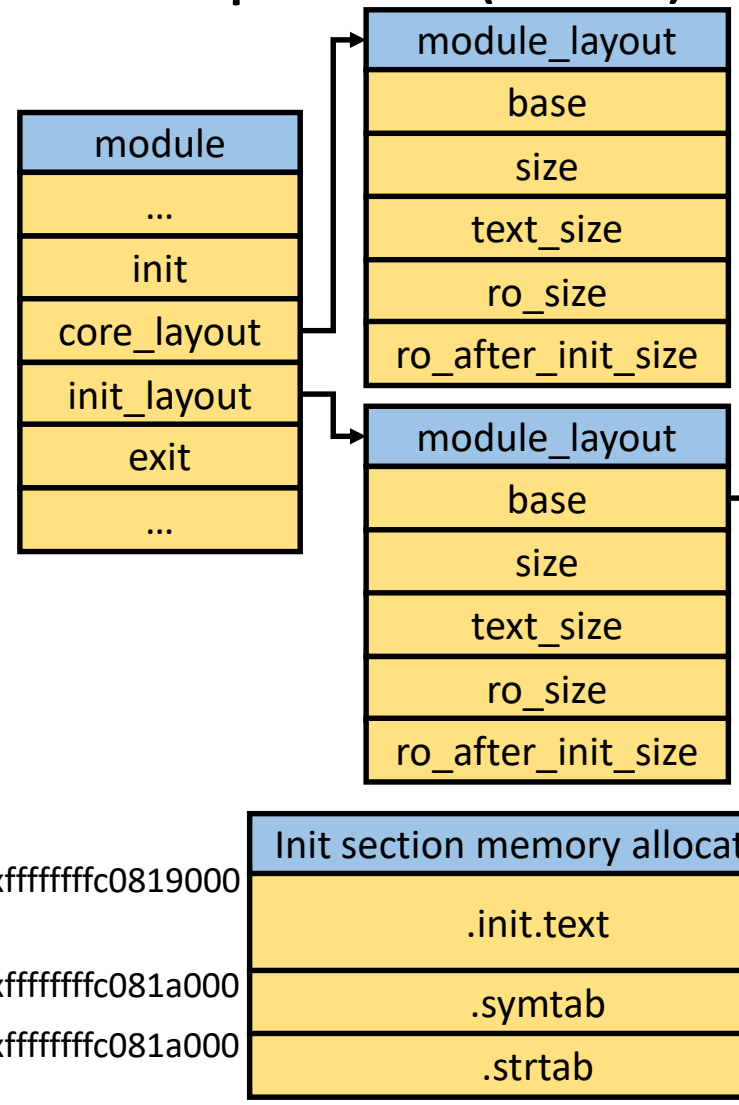




Update sh_addr (virtual address) of each section header table based on address ‘A’

Deep Dive into call path (4/7)

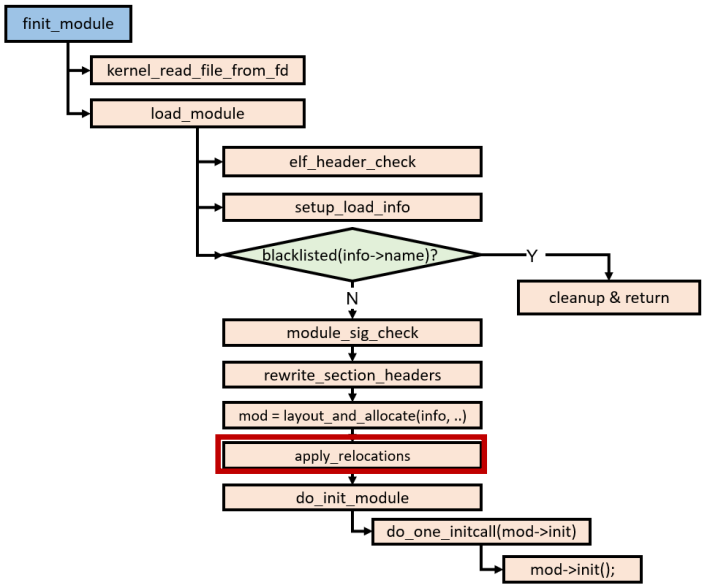


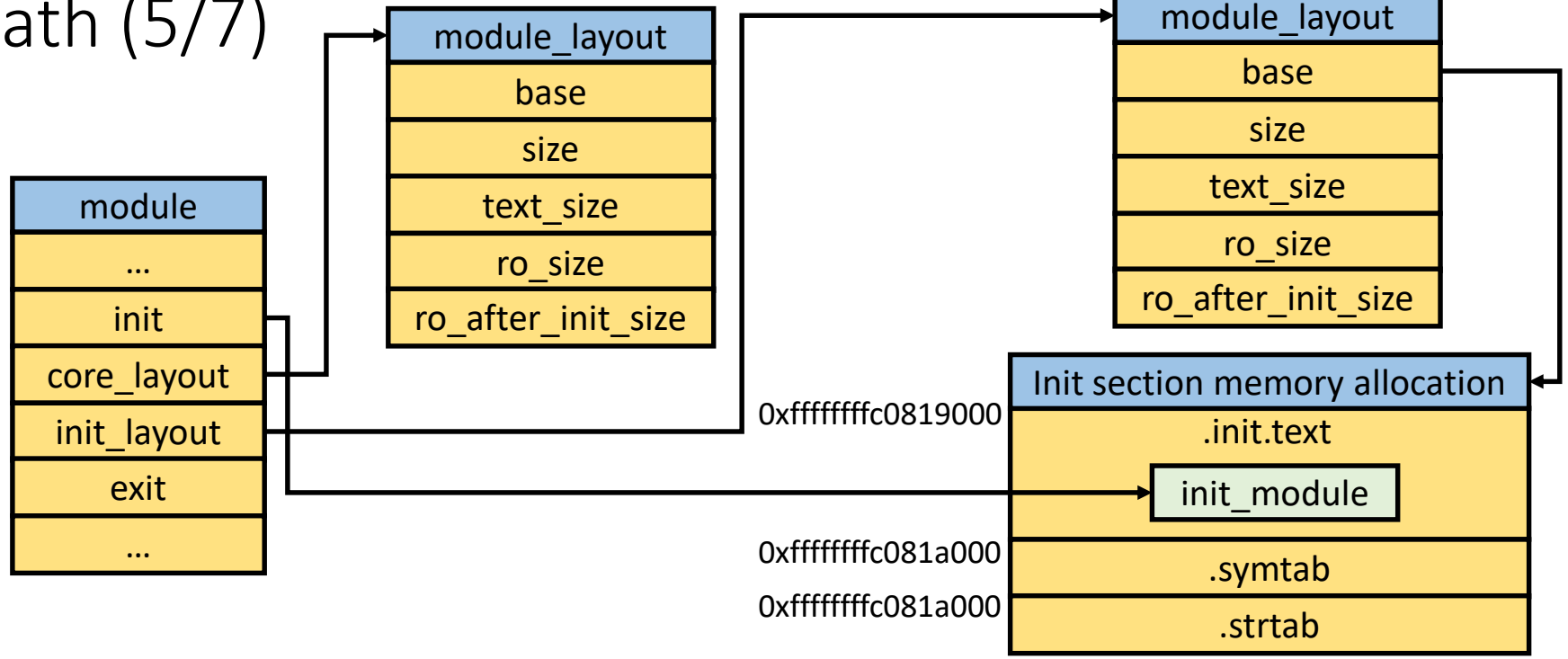



```
Core section allocation order:
 .text
 .exit.text
 .note.gnu.build-id
 .note.Linux
 .rodata.str1.1
 .rodata.str1.8
 __mcount_loc
 .orc_unwind_ip
 .orc_unwind
 .data
 .gnu.linkonce.this_module
 .bss
Init section allocation order:
 .init.text
 .symtab
 .strtab
final section addresses:
 0xffffffffc0815000 .note.gnu.build-id
 0xffffffffc0815024 .note.Linux
 0xffffffffc0814000 .text
 0xffffffffc0819000 .init.text
 0xffffffffc0814000 .exit.text
 0xffffffffc081503c .rodata.str1.1
 0xffffffffc0815058 .rodata.str1.8
 0xffffffffc0815078 __mcount_loc
 0xffffffffc0815080 .orc_unwind_ip
 0xffffffffc0815090 .orc_unwind
 0xffffffffc0816000 .data
 0xffffffffc0816000 .gnu.linkonce.this_module
 0xffffffffc08163c0 .bss
 0xffffffffc081a000 .symtab
 0xffffffffc081a450 .strtab
```

Update sh_addr (virtual address) of each section header table based on core/init section memory allocation

Deep Dive into call path (5/7)

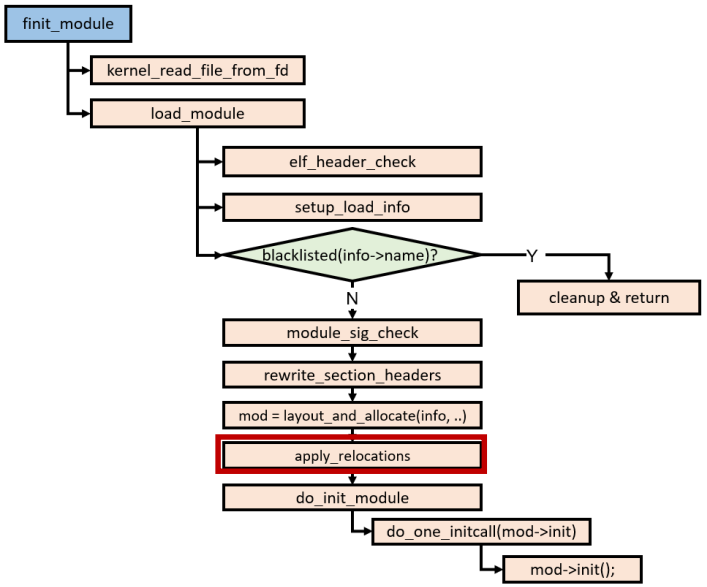


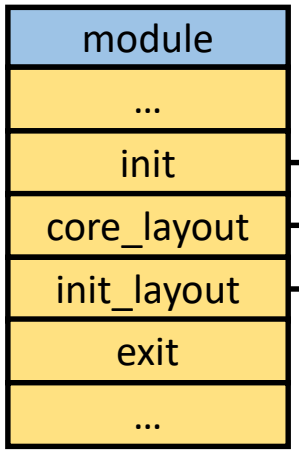


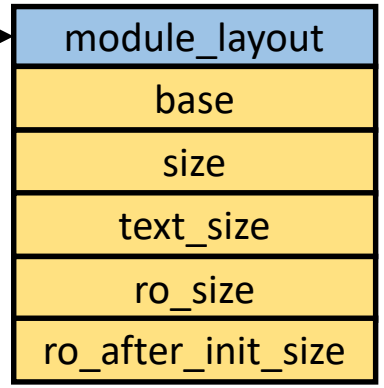

```
crash> struct module -o -x
struct module {
    [0x0] enum module_state state;
    [0x8] struct list_head list;
   [0x18] char name[56];
   [0x50] struct module_kobject mkobj;
   ...
  [0x150] int (*init)(void);
  ...
  [0x380] void (*exit)(void);
  [0x388] atomic_t refcnt;
  [0x390] struct error_injection_entry *ei_funcs;
  [0x398] unsigned int num_ei_funcs;
}
SIZE: 0x3c0
```

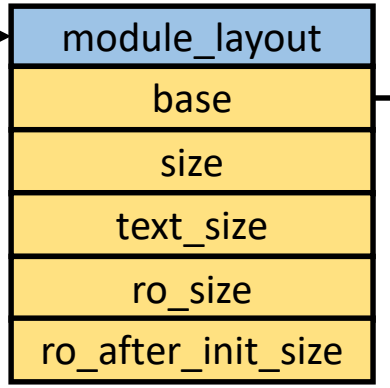
```
# readelf -a hello.ko

Section Headers:
  [Nr] Name              Type             Address           Offset
       Size              EntSize          Flags  Link  Info  Align
  ...
  [18] .gnu.linkonce.thi PROGBITS         0000000000000000  00000300
       00000000000003c0  0000000000000000  WA       0     0     64
  [19] .rela.gnu.linkonc RELA             0000000000000000  0001f178
       0000000000000030  0000000000000018   I      37    18     8
...
Relocation section '.rela.gnu.linkonce.this_module' at offset 0x1f178 contains 2
 entries:
  Offset          Info           Type           Sym. Value    Sym. Name + Addend
000000000150  002b00000001 R_X86_64_64       0000000000000000 init_module + 0
000000000380  002900000001 R_X86_64_64       0000000000000000 cleanup_module + 0
```

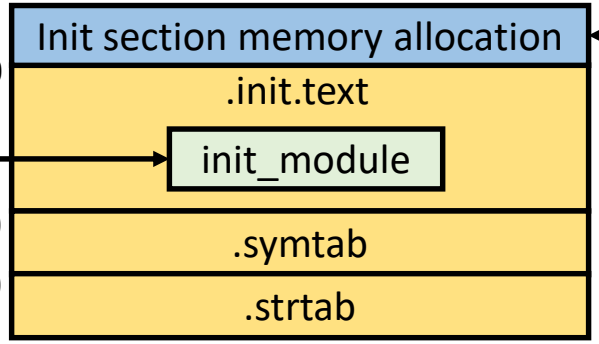
Deep Dive into call path (6/7)







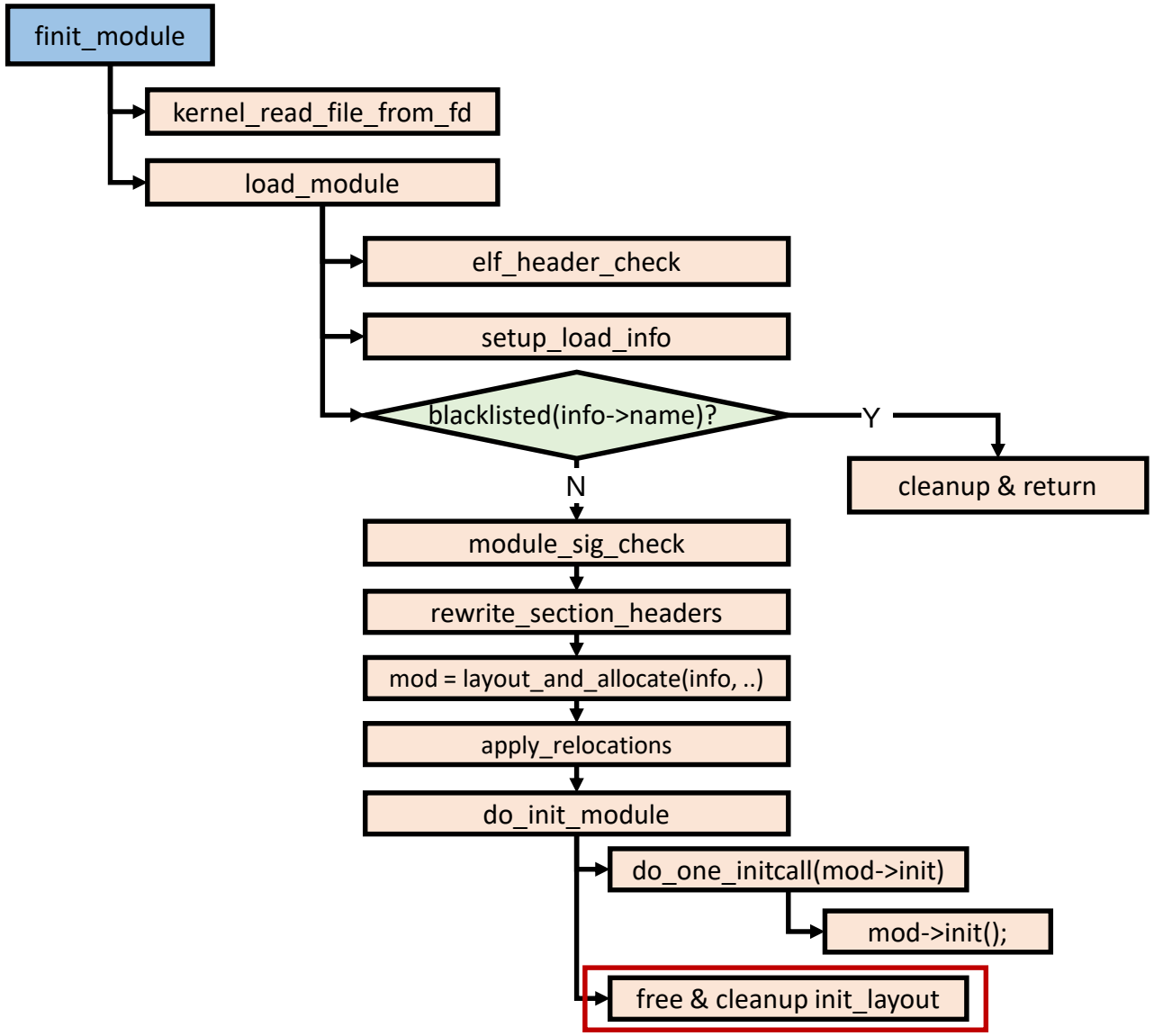





```
final section addresses:
 0xffffffffc0815000 .note.gnu.build-id
 0xffffffffc0815024 .note.Linux
 0xffffffffc0814000 .text
 0xffffffffc0819000 .init.text
 0xffffffffc0814000 .exit.text
 0xffffffffc081503c .rodata.str1.1
 0xffffffffc0815058 .rodata.str1.8
 0xffffffffc0815078 __mcount_loc
 0xffffffffc0815080 .orc_unwind_ip
 0xffffffffc0815090 .orc_unwind
 0xffffffffc0816000 .data
 0xffffffffc0816000 .gnu.linkonce.this_module
 0xffffffffc08163c0 .bss
 0xffffffffc081a000 .symtab
 0xffffffffc081a450 .strtab
```

```
crash> struct module 0xffffffffc0816000
struct module {
  ...
  name = "hello\000\000\000\000\000\000\
000\000\000\000\000\000\000\000\000\000\
000\000\000\000\000\000\000\000\000\000\
000\000\000\000\000\000\000\000\000\000\
000\000\000\000\000\000\000\000\000\000\
000\000\000\000",
  ...
  init = 0xffffffffc0819000,
  ...
  exit = 0xffffffffc0814000,
  refcnt = {
    counter = 1
  },
  ei_funcs = 0x0,
  num_ei_funcs = 0
}
```

Deep Dive into call path (7/7)




```
#define __init  __section(".init.text")

static int __init hello_init(void)
{
        ...
}

--> after expansion

static int__section(".init.text") hello_init(void)
{
        ...
}
```

```
crash> struct module 0xffffffffc0816000
struct module {
  ...
  init = 0xffffffffc0819000,
  core_layout = {
    base = 0xffffffffc0814000,
    ...
  },
  init_layout = {
    base = 0x0,
    size = 0,
    text_size = 0,
    ro_size = 0,
    ro_after_init_size = 0,
    ...
  },
  ...
}
```

Free memory space of init_layout after calling mod->init()

modinfo

```
# readelf -a hello.ko
Section Headers:
  [Nr] Name              Type             Address           Offset
       Size              EntSize          Flags  Link  Info  Align
  ...
  [12] .modinfo          PROGBITS         0000000000000000  00000108
       00000000000000a9  0000000000000000   A       0     0     1
  ...
```

```
# xxd hello.ko
...
00000100: 0000 0000 0000 0000 6c69 6365 6e73 653d  ........license=
00000110: 4750 4c00 6175 7468 6f72 3d41 6472 6961  GPL.author=Adria
00000120: 6e00 6465 7363 7269 7074 696f 6e3d 4865  n.description=He
00000130: 6c6c 6f20 576f 726c 6420 2121 0073 7263  llo World !!.src
00000140: 7665 7273 696f 6e3d 4635 3437 4336 3944  version=F547C69D
00000150: 4333 3437 4445 3434 3944 4431 4343 3200  C347DE449DD1CC2.
00000160: 6465 7065 6e64 733d 0072 6574 706f 6c69  depends=.retpoli
00000170: 6e65 3d59 006e 616d 653d 6865 6c6c 6f00  ne=Y.name=hello.
00000180: 7665 726d 6167 6963 3d35 2e31 302e 302d  vermagic=5.10.0-
00000190: 7263 372b 2053 4d50 206d 6f64 5f75 6e6c  rc7+ SMP mod_unl
000001a0: 6f61 6420 6d6f 6476 6572 7369 6f6e 7320  oad modversions
...
```

```
# modinfo hello.ko
filename:       /root/adrian/hello_world/hello.ko
license:        GPL
author:         Adrian
description:    Hello World !!
srcversion:     F547C69DC347DE449DD1CC2
depends:
retpoline:      Y
name:           hello
vermagic:       5.10.0-rc7+ SMP mod_unload modversions
```

Key=Value format in .modinfo section